



Figure 16-2: A `textYesNoDialog` used as an Alert dialog

Custom Version

Setting up a custom Yes/No dialog is more complicated but allows for much more control. Here is the interface of the `customYesNoDialog()` function:

```
customYesNoDialog(theWindow, oDialogImage, oPromptText, oYesButton,  
                  oNoButton)
```

Before you can call this function, you need to create objects for the background of the dialog, the prompt text, and the Yes and No buttons. You would typically use `Image`, `DisplayText`, and `CustomButton` (or `TextButton`) objects created from `pygwidgets` classes for this purpose. The `customYesNoDialog()` code demonstrates polymorphism by calling the `handleEvent()` method of the buttons, so it doesn't matter whether you use `CustomButtons` or `TextButtons`, and by calling the `draw()` method of all objects that make up the dialog. Because you create all these objects, you can customize the look of any or all of them. You will need to supply your own artwork for any `Image` and `CustomButton` objects and customarily place them in the *images* folder of the project.

When implementing a custom Yes/No dialog, typically you would write an intermediate function like `showCustomYesNoDialog()`, shown in [Listing 16-1](#). Then, at the place in your code where you want to show the dialog, rather than calling `customYesNoDialog()` directly, you instead call the intermediate function, which both instantiates the widgets and makes the actual call.

```
def showCustomYesNoDialog(theWindow, theText):  
    ❶ oDialogBackground = pygwidgets.Image(theWindow, (60, 120),  
                                           'images/dialog.png')  
    ❷ oPromptDisplayText = pygwidgets.DisplayText(theWindow, (0, 170),
```